

Ministral-3 3B를 3비트로 압축하다

30억 개 파라미터를 가진 언어 모델을 **FOEM** 기법으로 3비트까지 압축했습니다. 이 문서는 양자화가 무엇인지 모르는 사람도 따라올 수 있도록, 개념부터 결과 해석까지 차근차근 설명합니다.

원본 모델 Ministral-3-3B-Instruct (BF16) 기법 FOEM · 3-bit · group 128 작성일 2026-06-17

2.7GB

최종 모델 크기
(원본 ~6.0GB)

2.8×

전체 모델 압축률
(선형층은 5.3×

0/182

실패(RTN 폴백)
모든 층 정상 처리

10.87

PPL · WikiText-2
(낮을수록 좋음)

그래서 "양자화"가 뭔가요?

언어 모델은 수십억 개의 **가중치(weight)**라는 숫자로 이루어져 있습니다. 원래 이 숫자들은 하나당 16비트 (BF16)로 정밀하게 저장되는데, 그만큼 용량이 크고 무겁습니다. **양자화(Quantization)**는 이 숫자들을 더 적은 비트 수 — 이번엔 **3비트** — 로 거칠게 근사해서 저장하는 기술입니다.

쉽게 말하면

고해상도 사진을 **색 수를 줄여(256색 등)** 용량을 확 줄이는 것과 비슷합니다. 파일은 가벼워지지만 색이 살짝 뭉개지죠. 모델 양자화도 똑같습니다 — **크기는 작아지지만 약간의 정밀도(품질)를 내줍니다.** 잘 만든 양자화는 이 손실을 최소로 누르는 게 핵심입니다.

16비트로는 6만 개 이상의 값을 표현할 수 있지만, 3비트로는 단 **8개**의 값만 표현할 수 있습니다. 이 8칸 안에 원본의 분포를 최대한 똑똑하게 옥여넣는 것이 양자화 알고리즘이 하는 일이며, 이번에 사용한 알고리즘이 **FOEM**입니다.

이번에 무엇을 했나

모델 · 방식		실행 환경	
원본	Ministral-3 3B · BF16	GPU	RTX 5070 · 12GB
양자화 방법	FOEM	gptqmodel	7.1.0
비트	3-bit	transformers	5.12.1
group_size	128	torch	2.11.0 · cu128
attn_impl	sdpa	캘리브레이션	C4 · 256 샘플

개념 — 캘리브레이션

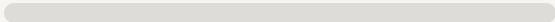
3비트의 8칸을 "어디에" 배치할지 정하려면, 모델이 실제로 어떤 숫자를 자주 쓰는지 관찰해야 합니다. 그래서 일반 텍스트 데이터(C4)의 256개 샘플을 모델에 흘려보내 활성값 분포를 측정했습니다. 이 과정을 **캘리브레이션**이라 하며, 학습이 아니라 "관찰"입니다.

얼마나 작아졌나

16비트를 3비트로 바꾸면 이론상 **5.33배**($16 \div 3$) 작아져야 합니다. 그런데 실제 전체 모델은 **2.8배**만 작아졌습니다. 왜 차이가 날까요?

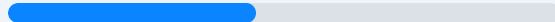
원본 (선형층)

~6.0 GB



양자화 후 (전체)

2.7 GB



이유는 두 가지입니다.

1. **전부 3비트가 되는 게 아닙니다.** 3비트로 압축된 건 **선형층 182개**뿐이고, 단어 임베딩·출력층·정규화 레이어 등은 품질을 위해 **BF16** 그대로 남겨둡니다.
2. **부가 정보 오버헤드.** `group_size=128`마다 복원에 필요한 `scale·zero` 값을 따로 저장하므로 **약 71MB**가 추가됩니다.

```
# group_size=128 오버헤드
scale (fp16): 3.02B / 128 × 2B ≈ 47 MB
zero (int8): 3.02B / 128 × 1B ≈ 24 MB
합계 ≈ 71 MB
```

품질은 어땠나 — 최종 점수와 loss

PPL · WIKITEXT-2

10.87

정확히 10.8654 · ↓ 낮을수록 좋음

개념 — PPL (퍼플렉서티)

모델이 다음 단어를 얼마나 잘 맞히는지 보는 종합 점수로, **낮을수록 좋습니다**. 값이 10.87이라는 건 "평균적으로 다음 단어 후보를 약 11개로 좁혔다"는 의미입니다. 표준 벤치마크 **WikiText-2**(시퀀스 2048 · 슬라이딩 윈도우 147개)에서 1.3분 만에 측정했으며, **양자화 전후 PPL 차이가 작을수록** 압축으로 인한 품질 손실이 적다는 뜻입니다.

PPL이 모델 **전체**의 성적표라면, 아래 **loss**는 층 하나하나의 양자화가 얼마나 잘 됐는지 보는 현미경입니다. 읽는 법을 먼저 짚고 갑니다.

개념 — LOSS 란?

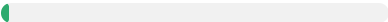
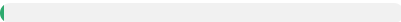
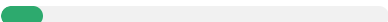
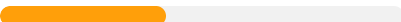
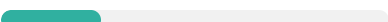
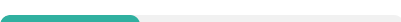
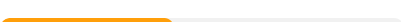
로그에 찍히는 **loss**는 가중치 숫자가 얼마나 변했는지가 아닙니다. 같은 입력을 넣었을 때 **원본과 양자화 모델의 실제 출력**이 얼마나 달라졌는지를 잴 값입니다.

$$\text{loss} = \| W \cdot X - W_q \cdot X \|^2 \quad (\text{낮을수록 원본과 비슷})$$

중요한 함정: 출력 차이를 재기 때문에 **활성값(X)이 큰 층은 loss도 덩달아 커집니다**. 그래서 절대 숫자가 크다고 무조건 양자화가 나쁜 게 아닙니다.

모듈별 평균 손실 — 두 가지 잣대로 비교

왼쪽은 층 전체의 **합산 절대 loss**, 오른쪽은 **가중치 1개당 loss**(절대값 ÷ 파라미터 수). 잣대를 바꾸면 순위가 뒤집힙니다.

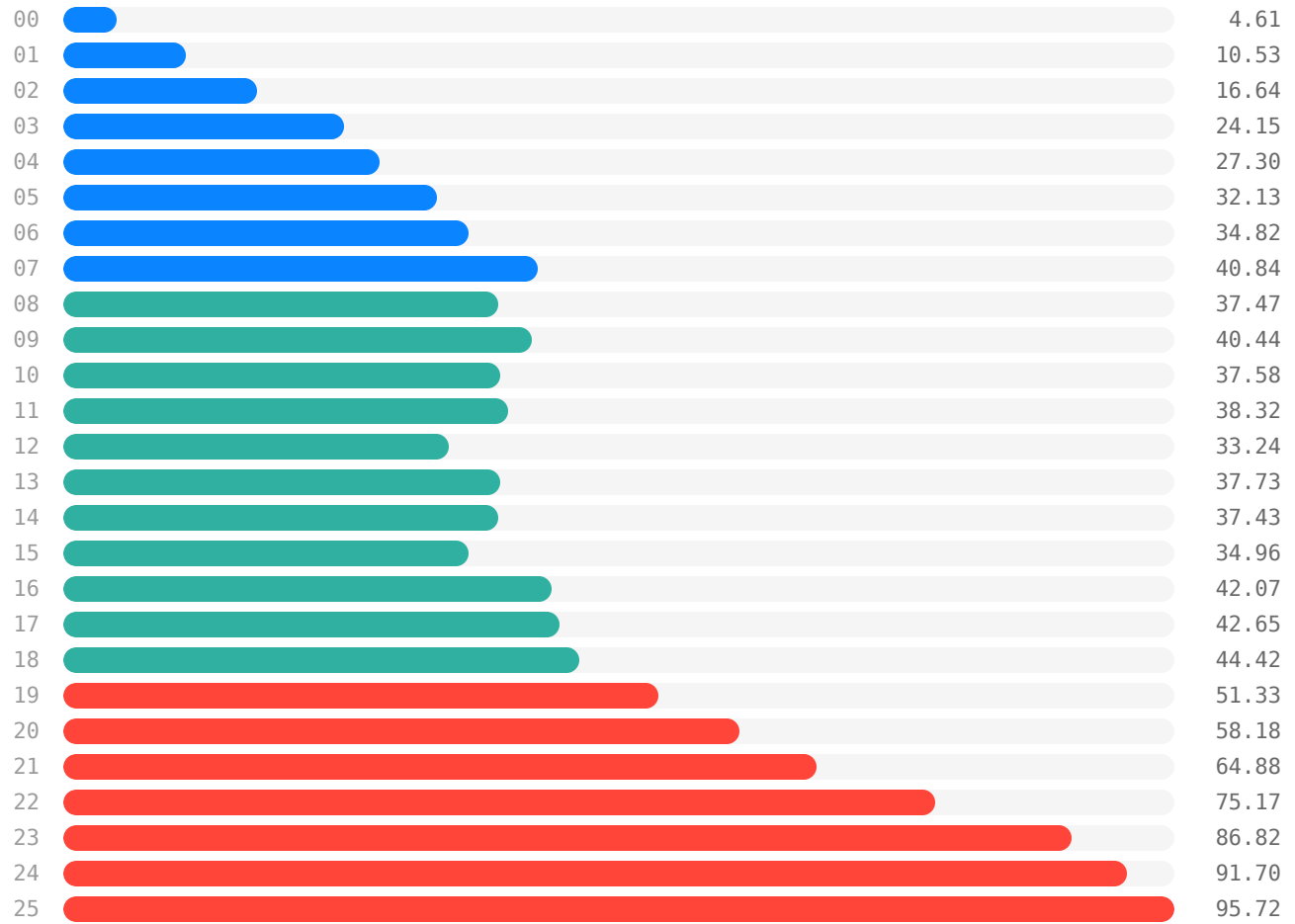
모듈	절대 평균 LOSS	파라미터당 LOSS ($\times 10^{-6}$)
self attn.o proj	 0.39	 0.03
mlp.down proj	 2.26	 0.08
self attn.v proj	 11.27	 3.58
self attn.k proj	 27.32	 8.68
self attn.q proj	 74.89	 5.95
mlp.up proj	 85.61	 3.02
mlp.gate proj	 105.47	 3.73

순위가 뒤집힙니다 — 절대값에 속지 마세요

절대 기준 1·2위는 **gate_proj·up_proj**지만, 파라미터당으로 보면 순위가 뒤집혀 **k_proj·q_proj**가 가장 높습니다. gate_proj는 파라미터가 28M으로 많아 합이 커 보일 뿐이고, k_proj는 3M밖에 안 되는데도 **가중치 하나 하나가 더 크게 망가졌다**는 뜻입니다. → 절대 숫자가 크다고 무조건 나쁜 양자화가 아닙니다.

레이어별 평균 손실 — 깊을수록 커진다

■ 초기 ■ 중간 ■ 후기



왜 어떤 부분은 양자화에 약한가

손실이 큰 모듈(`gate_proj`·`up_proj`)과 작은 모듈(`o_proj`·`down_proj`)에는 뚜렷한 규칙이 있습니다. **입력보다 출력 차원이 커지는 "확장형" 층일수록 손실이 큼**니다.

확장형 ↗ — 손실 큼

`gate/up_proj · q_proj`

예) 3072 → 9216 차원으로 펼침. 출력 항이 많아 합산 손실이 커지고, SwiGLU 게이트의 비선형성이 작은 오차도 증폭시킵니다.

축소형 ↘ — 손실 작음

`down_proj · o_proj · v_proj`

예) 9216 → 3072 차원으로 모음. 중요한 성분만 선별해 압축하는 역할이라 양자화 오차에 본래 강인합니다.

개념 — SWIGLU

최신 LLM의 MLP는 `gate_proj`의 출력을 일종의 "문(gate)"으로 써서 `up_proj`의 신호를 통과시킬지 조절합니다. 문 역할이라 값이 살짝만 틀어져도 통과량이 크게 흔들리고, 그래서 `gate_proj`가 가장 민감합니다.

깊은 레이어일수록 더 어렵다

위 차트에서 봤듯 0번 층(4.6)에서 25번 층(95.7)으로 갈수록 손실이 **20배 넘게** 커집니다. 모델의 역할을 3구간으로 나눠 보면 이유가 분명합니다.

■ 초기 · 0-7

4.6→41

기본 어휘·문법 패턴 추출. 분포가 단순해 양자화가 쉽습니다.

■ 중간 · 8-18

37→44

중간 단계 추상화. 손실이 평평하게 유지되는 안정 구간.

■ 후기 · 19-25

51→96

추론·맥락 이해 담당. 값의 분포가 넓어 가장 어렵습니다.

초기 층은 활성값 분포가 단순하고 안정적이라 3비트 8칸으로도 충분합니다. 반면 "추론"을 담당하는 후기 층은 표현해야 할 값의 범위가 훨씬 넓어 8칸에 담기 빠듯하죠. 이것이 **3비트가 4비트 대비 후기 층에서 품질 차이를 크게 만드는** 근본 원인입니다.

FOEM은 어떻게 손실을 줄이나

FOEM(First-Order Error Matters, AAAI 2026)은 가중치를 한 열씩 양자화하면서 생기는 오차를 똑똑하게 처리하는 GPTQ 계열 기법입니다.

$$\Delta W = -e_i \otimes H^{-1}[i,:] \text{ ① 기본 GPTQ 보정} \\ - (W - W_{fp}) \cdot H^{-2} \cdot \beta \text{ ② FOEM 오차 피드백 } (\beta=0.2)$$

① GPTQ 항

한 열을 양자화하며 생긴 오차를 Hessian 역행렬로 나머지 열에 분산시켜 미리 보상합니다.

② FOEM β 항

지금까지 쌓인 누적 오차를 직접 되먹임해 그 자리에서 흡수합니다. 오차가 뒤로 전파되지 않습니다.

이번 실험은 **alpha=0**(1차 보정 GPTAQ 비활성), **beta=0.2**(오차 피드백 활성) 설정이라 ② **β 항만 동작했습니다.** 그 결과 182개 모듈 전부 풀백 없이 처리됐습니다.

알아두면 좋은 설정값

damp_percent = 0.05

Hessian 역행렬 계산이 수치적으로 폭발하지 않도록 대각선에 살짝 더해주는 안전장치입니다. 너무 작으면 계산이 터져 RTN 폴백(거친 단순 양자화)이 일어나고, 너무 크면 보정 방향이 왜곡됩니다. **0.05**는 업계 표준값으로, 이번 실험에선 RTN 폴백이 **0건**이라 완벽히 작동했습니다.

group_size = 128

가중치 128개마다 독립적인 scale·zero 값을 뒀서 그 작은 묶음 안에서 최적 스케일을 찾습니다. 숫자가 작을수록 정밀하지만 오버헤드가 커집니다 — 128은 품질과 크기의 표준 절충점입니다.

GROUP_SIZE	오버헤드	품질	용도
32	높음	최상	고품질 우선
128	중간	양호	표준 (이 실험)
256	낮음	보통	크기 우선
-1	최대	최상	연구용

정리하면

- 30억 파라미터 모델을 **2.7GB(전체 2.8배 압축)**로 줄였고, 182개 선형층 **전부 풀백 없이** 양자화 성공.
- 손실은 **확장형 층(gate/up_proj)**과 **후기 레이어**에 집중 — 구조적으로 예상되는 정상 패턴.
- 3비트는 후기 추론 층에서 부담이 크므로, 품질이 중요하다면 **4비트** 또는 민감 층만 고비트로 두는 혼합 방식을 검토할 만함.

사용 방법

```
from gptqmodel import GPTQModel

model = GPTQModel.from_quantized(
    ".../Ministral-3-3B-Instruct-2512-BF16_foem_3bit"
)
```

용어 한눈에

BF16 — 가중치를 16비트로 저장하는 원본 형식.

캘리브레이션 — 양자화 범위를 정하러 데이터를 흘려 분포를 관찰하는 과정.

loss — 원본과 양자화 모델의 출력 차이. 낮을수록 좋음.

3-bit — 값을 8단계로만 표현하는 압축 형식.

Hessian — 활성값으로 만든 행렬(XX). 오차 보정 방향의 기준.

RTN 풀백 — 정교한 보정이 실패해 단순 반올림으로 대체되는 것.